

1. 题目深度解构

题意重述

给一棵有 n 个点的树，每个点是黑色或白色， $s_i = 0$ 表示黑点， $s_i = 1$ 表示白点。

一次跳跃可以从点 x 跳到任意满足树上距离 $\text{dist}(x, y) \leq k$ 的点 y 。

对每个目标点 $2, 3, \dots, n$ ，从点 1 出发，问最少需要“落到黑点”多少次才能到达该目标点。起点 1 不算“落到”。

把问题抽象成图论模型：

构造一张隐式图 G ：

- 树上任意两点 x, y ，若 $\text{dist}(x, y) \leq k$ ，则在 G 中可从 x 走到 y ；
- 走到点 y 的代价为 $\text{cost}(y)$ ，其中黑点代价 1 ，白点代价 0 ；
- 求从 1 到所有点的最短路。

约束分析

$n \leq 5 * 10^5$ ，且所有测试数据的 n 总和不超过 $5 * 10^5$ 。

如果暴力建出隐式图，最坏情况下 $k \geq n$ ，任意两点都能互相跳，边数是 $O(n^2)$ ，完全不可行。

因此目标复杂度应当是：

$$O(n \log n)$$

或最多接近：

$$O(n \log^2 n)$$

题目给的是树，并且跳跃条件是树上距离不超过 k ，这是核心突破口。我们需要在树上快速处理“半径为 k 的球”。

题目类型判定

这题属于：

隐式图最短路 + 0-1 BFS + 点分治 / 重心分解

本质是最短路，但难点不在最短路本身，而在于如何高效枚举：

$$\{v \mid \text{dist}(u, v) \leq k\}$$

也就是树上距离球。

2. 思维路径推导

从暴力开始

最直接的想法是：

对每个点 u ，找出所有 $\text{dist}(u, v) \leq k$ 的点 v ，连一条边，然后跑最短路。

但如果 k 很大，图会接近完全图，边数达到 $O(n^2)$ ，无法接受。

因为落到白点代价是 0，落到黑点代价是 1，所以边权只有 0/1，最短路可以用 0-1 BFS。

可是 0-1 BFS 仍然需要“枚举出边”。所以真正的问题变成：

当我们从某个点 u 扩展时，如何快速找出所有还没确定答案、且距离 u 不超过 k 的点？

为什么不能只在根到目标路径上 DP？

一个容易误判的方向是：目标是从 1 到 x ，是否只需要考虑树上路径 $1 \rightarrow x$ ？

不行。

样例 2 中，到点 8 时，可以走：

$$1 \rightarrow 5 \rightarrow 8$$

其中 5 并不在 $1 \rightarrow 8$ 的树上路径上，但它是白点，作为中转点可以减少黑点落脚次数。

所以最优路径可能绕到目标路径外的白点上，不能做简单的树形 DP。

关键观察 1：第一次被找到时答案就是最优

设 $d[u]$ 是已经确定的最小黑点落脚次数。

从 u 可以跳到所有 $\text{dist}(u, v) \leq k$ 的点，且新代价为：

$$d[u] + [v \text{ 是黑点}]$$

注意，代价只和目标点 v 的颜色有关，和边本身无关。

如果我们按照 0-1 BFS 的顺序弹点，则当前弹出的 u 的 $d[u]$ 不会大于之后弹出的点。于是当一个还没被确定的点 v 第一次被某个 u 找到时：

- 之后再从别的点 w 找到它，一定有 $d[w] \geq d[u]$ ；
- 而落到 v 的代价 $[v \text{ 是黑点}]$ 是固定的；

因此之后不可能得到更小答案。

所以每个点只需要被确定一次。

这让问题变成了“report and delete”：

每次查询一个树上距离球，把其中尚未确定的点全部取出来，并标记为已确定。

关键观察 2：用点分治拆树上距离球

对树做重心分解。

对每个点 u ，记录它在重心树上的所有祖先重心 c ，以及原树距离：

$$dist(u, c)$$

考虑一个点 v 是否满足：

$$dist(u, v) \leq k$$

在重心分解中， u 和 v 的重心树路径有一个最低公共重心 c 。这个 c 会把 u 和 v 分到不同的子块里，因此在原树上， c 位于 u 到 v 的路径上。

于是：

$$dist(u, v) = dist(u, c) + dist(c, v)$$

所以如果 v 在 u 的半径 k 球内，那么对于某个 u 的重心祖先 c ，有：

$$dist(c, v) \leq k - dist(u, c)$$

反过来，如果某个重心祖先 c 满足：

$$dist(u, c) + dist(c, v) \leq k$$

由三角不等式可知：

$$dist(u, v) \leq k$$

因此：

$$B(u, k) = \bigcup_{c \in \text{Anc}_{\text{centroid}}(u)} \{v \mid dist(c, v) \leq k - dist(u, c)\}$$

这就是本题的核心。

关键观察 3：每个重心维护一个按距离排序的桶

对每个重心 c ，维护一个数组：

$$bag[c] = \{(dist(c, v), v)\}$$

其中 v 是该重心分解阶段所在连通块内的所有点。

因为查询时只需要取出：

$$dist(c, v) \leq R$$

所以如果 $bag[c]$ 按距离非降序排列，就可以用一个指针 $ptr[c]$ 从前往后扫。

每个 (c, v) 只会被 $ptr[c]$ 扫过一次，总扫描量是：

$$O(n \log n)$$

代码里构造 $bag[c]$ 时，用 BFS 从重心 c 出发收集点，因此天然就是按距离非降序的，不需要排序。

最终算法

1. 对原树做重心分解。
2. 对每个点 u ，记录所有 $(c, dist(u, c))$ 。
3. 对每个重心 c ，维护 $bag[c] = \{(dist(c, v), v)\}$ ，按距离非降序。
4. 从点 1 开始做 0-1 BFS。
5. 弹出点 u 时：
 - 枚举 u 的所有重心祖先 (c, du) ；
 - 若 $du \leq k$ ，则需要取出 $bag[c]$ 中所有 $dist(c, v) \leq k - du$ 的点；
 - 对尚未确定的点 v ：
 - $d[v] = d[u] + (s[v] == '0')$
 - 若 v 是白点，放入队首；
 - 若 v 是黑点，放入队尾。
6. 输出 $d[2], d[3], \dots, d[n]$ 。

复杂度：

$$O(n \log n)$$

空间复杂度：

$$O(n \log n)$$

3. Trick 与陷阱

Trick 点拨

这题的关键 Trick 是：

把树上半径球查询转化为重心祖先上的若干个“按距离取前缀”。

对每个重心 c ，所有点按照 $dist(c, v)$ 放在一个桶中。查询 $B(u, k)$ 时，只需要枚举 u 的 $O(\log n)$ 个重心祖先，每个桶扫一个前缀。

另一个 Trick 是：

不需要真正从所有桶中删除点。

每个点第一次被确定答案后， $dis[v] \neq -1$ 。之后即使在别的桶里又扫到它，直接跳过即可。每个桶自己的指针单调前进，因此总复杂度仍然是 $O(n \log n)$ 。

易错点

1. `s_i = 0` 才是黑点，代价是 1；`s_i = 1` 是白点，代价是 0。
2. 起点 1 的颜色不计入答案，初始化 `dis[1] = 0`。
3. `n = 1` 时需要输出一个空行。
4. 重心分解如果递归写 DFS，链状树上可能爆栈；下面代码使用迭代写法。
5. 构造 `bag[c]` 时要按距离非降序。代码中用队列式 BFS 收集，因此不需要再排序。

4. 代码实现 (C++23, jiangly style)

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int T;
    cin >> T;
    while (T--) {
        int n, k;
        cin >> n >> k;

        string s;
        cin >> s;

        vector<vector<int>>> adj(n);
        for (int i = 1; i < n; i++) {
            int u, v;
            cin >> u >> v;
            --u, --v;
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        int lg = 1;
        while ((1 << lg) <= n) lg++;

        vector<vector<pair<int, int>>>> up(n), bag(n);
        for (auto &v : up) v.reserve(lg + 1);

        vector<int> par(n), sz(n), ptr(n), dis(n, -1);
        vector<char> dead(n);

        vector<int> roots, ord, st;
        vector<array<int, 3>> q;
        roots.reserve(n);
```

```

ord.reserve(n);
st.reserve(n);
q.reserve(n);

roots.push_back(0);

while (!roots.empty()) {
    int r = roots.back();
    roots.pop_back();

    ord.clear();
    st.clear();

    par[r] = -1;
    st.push_back(r);

    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        ord.push_back(u);

        for (auto v : adj[u]) {
            if (dead[v] || v == par[u]) continue;
            par[v] = u;
            st.push_back(v);
        }
    }

    int tot = ord.size();

    for (int i = tot - 1; i >= 0; i--) {
        int u = ord[i];
        sz[u] = 1;

        for (auto v : adj[u]) {
            if (!dead[v] && par[v] == u) {
                sz[u] += sz[v];
            }
        }
    }

    int c = -1;
    for (auto u : ord) {
        int mx = tot - sz[u];

        for (auto v : adj[u]) {
            if (!dead[v] && par[v] == u) {
                mx = max(mx, sz[v]);
            }
        }

        if (2 * mx <= tot) {
            c = u;
        }
    }
}

```

```

        break;
    }
}

bag[c].reserve(tot);

q.clear();
q.push_back({c, -1, 0});

for (int i = 0; i < int(q.size()); i++) {
    auto [u, p, d] = q[i];

    up[u].push_back({c, d});
    bag[c].push_back({d, u});

    for (auto v : adj[u]) {
        if (dead[v] || v == p) continue;
        q.push_back({v, u, d + 1});
    }
}

dead[c] = 1;

for (auto v : adj[c]) {
    if (!dead[v]) {
        roots.push_back(v);
    }
}
}

deque<int> que;
dis[0] = 0;
que.push_back(0);

while (!que.empty()) {
    int u = que.front();
    que.pop_front();

    for (auto [c, du] : up[u]) {
        if (du > k) continue;

        int r = k - du;
        auto &b = bag[c];
        int &p = ptr[c];

        while (p < int(b.size()) && b[p].first <= r) {
            int v = b[p++].second;

            if (dis[v] != -1) continue;

            dis[v] = dis[u] + (s[v] == '0');

            if (s[v] == '0') {

```

```

        que.push_back(v);
    } else {
        que.push_front(v);
    }
}
}

for (int i = 1; i < n; i++) {
    if (i > 1) cout << ' ';
    cout << dis[i];
}
cout << '\n';
}

return 0;
}

```

5. 总结与启示

难度评分

预估 Codeforces Rating:

2500

主要难点在于把“树上距离不超过 k 的隐式邻居枚举”转化为可维护的结构。

核心启示

当隐式图的边由树上距离限制定义时，可以考虑用重心分解把“距离球”拆成若干个按重心距离维护的前缀。

同类习题推荐

1. **Codeforces 342E - Xenia and Tree**
经典重心分解维护树上最近点。
2. **SPOJ QTREE5 - Query on a tree**
动态点集 + 树上最近距离，也是重心分解经典题。
3. **Codeforces 786B - Legacy**
隐式图最短路，通过数据结构避免显式建出海量边。

你这个 TLE 的直接触发点在截图里已经暴露了：


```
n = 427670
k = 398399
s = 0000000000...
```

也就是**所有点都是黑点**。这种情况完全不需要点分治。

如果全是黑点，那么从 1 到点 v 的答案就是最少跳几次能到达 v ：

$$ans[v] = \left\lceil \frac{dist(1, v)}{k} \right\rceil$$

原因很简单：

- 每次跳跃最多让树上距离推进 k ，所以至少需要 $\lceil dist / k \rceil$ 次；
- 沿着 $1 \rightarrow v$ 的树上路径，每次跳 k 条边，最后不足 k 也跳一次，一定可以达到；
- 全是黑点，所以每次落脚都贡献 1。

所以原代码在这个测试上做完整点分治，是明显浪费。

另外，原代码还有一个性能隐患：`up` 和 `bag` 都用 `vector<vector<pair<int,int>>>` 存储，每个（点，重心祖先）信息被存了两份，且有大量小 `vector` 分配，常数和内存都偏大。下面这份代码做了两类修正：

1. 增加必要特判：

- 全白：全 0
- 全黑： $\lceil depth / k \rceil$
- $maxDepth \leq k$ ：从 1 可以直接跳到所有点
- $k == 1$ ：路径黑点前缀和

2. 点分治部分改成扁平数组：

- `cdist[level * n + u]` 存 u 到该层重心的距离；
- 所有 `bag` 合并成一个全局数组，用 `[L[c], R[c]]` 表示重心 c 的桶；
- 邻接表改成 CSR，减少内存和分配开销。

完整修正版如下。

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

int main() {
    cin.tie(nullptr) -> sync_with_stdio(false);

    int T;
    cin >> T;
    while (T--) {
        int n, k;
        cin >> n >> k;
        string s;
        cin >> s;
```

```

vector<pair<int, int>> edges;
edges.reserve(n - 1);
vector<int> deg(n);

for (int i = 1; i < n; i++) {
    int u, v;
    cin >> u >> v;
    --u, --v;
    edges.emplace_back(u, v);
    deg[u]++, deg[v]++;
}

vector<int> head(n + 1);
for (int i = 0; i < n; i++) {
    head[i + 1] = head[i] + deg[i];
}

vector<int> cur = head, to(2 * (n - 1));
for (auto [u, v] : edges) {
    to[cur[u]++] = v;
    to[cur[v]++] = u;
}

vector<pair<int, int>>().swap(edges);
vector<int>().swap(deg);
vector<int>().swap(cur);

auto out = [&](auto f) {
    for (int i = 1; i < n; i++) {
        if (i > 1) cout << ' ';
        cout << f(i);
    }
    cout << '\n';
};

bool all0 = true, all1 = true;
for (auto c : s) {
    all0 &= c == '0';
    all1 &= c == '1';
}

if (all1) {
    out([&](int) { return 0; });
    continue;
}

vector<int> dep(n, -1), ord;
ord.reserve(n);
dep[0] = 0;
ord.push_back(0);

for (int i = 0; i < n; i++) {

```

```

        int u = ord[i];
        for (int e = head[u]; e < head[u + 1]; e++) {
            int v = to[e];
            if (dep[v] == -1) {
                dep[v] = dep[u] + 1;
                ord.push_back(v);
            }
        }
    }

    if (all0) {
        out([&](int i) { return (dep[i] + k - 1) / k; });
        continue;
    }

    int mxDep = *max_element(dep.begin(), dep.end());
    if (mxDep <= k) {
        out([&](int i) { return int(s[i] == '0'); });
        continue;
    }

    vector<int>().swap(dep);

    if (k == 1) {
        vector<int> ans(n), par(n, -1);
        ord.clear();
        ord.push_back(0);

        for (int i = 0; i < n; i++) {
            int u = ord[i];
            for (int e = head[u]; e < head[u + 1]; e++) {
                int v = to[e];
                if (v == par[u]) continue;
                par[v] = u;
                ans[v] = ans[u] + (s[v] == '0');
                ord.push_back(v);
            }
        }

        out([&](int i) { return ans[i]; });
        continue;
    }

    int lg = 1;
    while ((1 << lg) <= n) lg++;
    int H = lg + 1;

    vector<int> cdist(1LL * H * n), bag;
    bag.reserve(1LL * H * n);

    vector<int> cp(n, -1), lev(n), L(n), R(n), ptr(n);
    vector<int> par(n), sz(n), dis(n, -1);
    vector<char> dead(n);

```

```

vector<tuple<int, int, int>> roots;
vector<int> st;
vector<array<int, 3>> q;

roots.reserve(n);
st.reserve(n);
q.reserve(n);

roots.emplace_back(0, -1, 0);

while (!roots.empty()) {
    auto [r, pc, lv] = roots.back();
    roots.pop_back();

    ord.clear();
    st.clear();

    par[r] = -1;
    st.push_back(r);

    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        ord.push_back(u);

        for (int e = head[u]; e < head[u + 1]; e++) {
            int v = to[e];
            if (!dead[v] && v != par[u]) {
                par[v] = u;
                st.push_back(v);
            }
        }
    }

    int tot = ord.size();

    for (int i = tot - 1; i >= 0; i--) {
        int u = ord[i];
        sz[u] = 1;

        for (int e = head[u]; e < head[u + 1]; e++) {
            int v = to[e];
            if (!dead[v] && par[v] == u) {
                sz[u] += sz[v];
            }
        }
    }

    int c = -1;
    for (auto u : ord) {
        int mx = tot - sz[u];
    }
}

```

```

        for (int e = head[u]; e < head[u + 1]; e++) {
            int v = to[e];
            if (!dead[v] && par[v] == u) {
                mx = max(mx, sz[v]);
            }
        }

        if (2 * mx <= tot) {
            c = u;
            break;
        }
    }

    cp[c] = pc;
    lev[c] = lv;
    L[c] = ptr[c] = bag.size();

    q.clear();
    q.push_back({c, -1, 0});

    int base = lv * n;

    for (int i = 0; i < int(q.size()); i++) {
        auto [u, p, d] = q[i];

        cdist[base + u] = d;
        bag.push_back(u);

        for (int e = head[u]; e < head[u + 1]; e++) {
            int v = to[e];
            if (!dead[v] && v != p) {
                q.push_back({v, u, d + 1});
            }
        }
    }

    R[c] = bag.size();
    dead[c] = 1;

    for (int e = head[c]; e < head[c + 1]; e++) {
        int v = to[e];
        if (!dead[v]) {
            roots.emplace_back(v, c, lv + 1);
        }
    }
}

deque<int> que;
dis[0] = 0;
que.push_back(0);

while (!que.empty()) {
    int u = que.front();

```

```

        que.pop_front();

    for (int c = u; c != -1; c = cp[c]) {
        int l = lev[c];
        int base = l * n;
        int du = cdist[base + u];

        if (du > k) continue;

        int r = k - du;
        int &p = ptr[c];

        while (p < R[c] && cdist[base + bag[p]] <= r) {
            int v = bag[p++];

            if (dis[v] != -1) continue;

            dis[v] = dis[u] + (s[v] == '0');

            if (s[v] == '0') {
                que.push_back(v);
            } else {
                que.push_front(v);
            }
        }
    }

    out([&](int i) { return dis[i]; });
}

return 0;
}

```

这版在你截图那个全黑大数据上会直接走 $O(n)$ 分支，不会再建点分治；一般情况仍然是 $O(n \log n)$ ，但内存布局和常数比上一版低很多。

